

AFRILANG Stdlib

std/c/gametree

Bibliothèque AFRILANG `std/c/gametree` (niveau complex, catégorie complex). Elle expose 8 fonction(s) : branchingFactor,

```
`import "std/c/gametree" · `use gametree`
```

- `branchingFactor(children list number) ? number`
- `treeDepth(depths list number) ? number`
- `minimaxLeaf(values list number, maxNode bool) ? number`
- `expectimax(values list number, probs list number) ? number`
- `alphaBetaWindow(alpha number, beta number) ? list number`
- `gameTreeSize(branching number, depth number) ? number`
- `plyCount(moves list number) ? number`
- `bestChild(scores list number) ? number`

Category: complex | Tier: complex | Functions: 8

FRANCAIS

1. Vue d'ensemble

Bibliothèque AFRILANG ``std/c/gametree`` (niveau complex, catégorie complex). Elle expose 8 fonction(s) : `branchingFactor`,

```
`import "std/c/gametree"` . `use gametree`  
- `branchingFactor(children list number) ? number`  
- `treeDepth(depths list number) ? number`  
- `minimaxLeaf(values list number, maxNode bool) ? number`  
- `expectimax(values list number, probs list number) ? number`  
- `alphaBetaWindow(alpha number, beta number) ? list number`  
- `gameTreeSize(branching number, depth number) ? number`  
- `plyCount(moves list number) ? number`  
- `bestChild(scores list number) ? number`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/gametree"  
use gametree
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés ? graphes, NLP, réseaux complexes.

3. Référence API

```
? branchingFactor(children list number) ? number  
? treeDepth(depths list number) ? number  
? minimaxLeaf(values list number, maxNode bool) ? number  
? expectimax(values list number, probs list number) ? number  
? alphaBetaWindow(alpha number, beta number) ? list  
? gameTreeSize(branching number, depth number) ? number  
? plyCount(moves list number) ? number  
? bestChild(scores list number) ? number
```

4. Exemples d'utilisation

```
import "std/c/gametree"  
use gametree  
  
create result = branchingFactor(/* args */)   
say result  
  
create result = treeDepth(/* args */)   
say result  
  
create result = minimaxLeaf(/* args */)   
say result  
  
create result = expectimax(/* args */)   
say result  
  
create result = alphaBetaWindow(/* args */)   
say result  
  
create result = gameTreeSize(/* args */)   
say result
```

5. Bonnes pratiques

```
? Préférez `import "std/c/gametree"` en tête de fichier.  
? Lancez `afrilang check` avant de livrer.  
? Combinez avec les modules stdlib de la même catégorie.  
? Voir /stdlib/ pour le source live et les démos playground.  
? Signalez les problèmes sur GitHub si une export manque.
```

6. Annexe ? source

```
module gametree
function branchingFactor(children list number) returns number
end
function treeDepth(depths list number) returns number
end
function minimaxLeaf(values list number, maxNode bool) returns number
end
function expectimax(values list number, probs list number) returns number
end
function alphaBetaWindow(alpha number, beta number) returns list number
end
function gameTreeSize(branching number, depth number) returns number
end
function plyCount(moves list number) returns number
end
function bestChild(scores list number) returns number
end
end
```

ENGLISH

1. Overview

AFRILANG library `std/c/gametree` (tier complex, category complex). Exports 8 function(s):
branchingFactor, treeDepth, m

```
`import "std/c/gametree"` . `use gametree`  
- `branchingFactor(children list number) ? number`  
- `treeDepth(depths list number) ? number`  
- `minimaxLeaf(values list number, maxNode bool) ? number`  
- `expectimax(values list number, probs list number) ? number`  
- `alphaBetaWindow(alpha number, beta number) ? list number`  
- `gameTreeSize(branching number, depth number) ? number`  
- `plyCount(moves list number) ? number`  
- `bestChild(scores list number) ? number`
```

2. Installation & import

Add the module to your program:

```
import "std/c/gametree"  
use gametree
```

Tier: complex · Category: Complex modules (c/)
Advanced domains ? graphs, NLP, complex networks.

3. API reference

```
? branchingFactor(children list number) ? number  
? treeDepth(depths list number) ? number  
? minimaxLeaf(values list number, maxNode bool) ? number  
? expectimax(values list number, probs list number) ? number  
? alphaBetaWindow(alpha number, beta number) ? list  
? gameTreeSize(branching number, depth number) ? number  
? plyCount(moves list number) ? number  
? bestChild(scores list number) ? number
```

4. Usage examples

```
import "std/c/gametree"  
use gametree  
  
create result = branchingFactor(/* args */)   
say result  
  
create result = treeDepth(/* args */)   
say result  
  
create result = minimaxLeaf(/* args */)   
say result  
  
create result = expectimax(/* args */)   
say result  
  
create result = alphaBetaWindow(/* args */)   
say result  
  
create result = gameTreeSize(/* args */)   
say result
```

5. Best practices

```
? Prefer `import "std/c/gametree"` at the top of your file.  
? Run `afrilang check` before shipping.  
? Combine with related stdlib modules from the same category.  
? See /stdlib/ for the live source and playground demos.  
? Report issues on GitHub if an export is missing or undocumented.
```

6. Appendix ? source

```
module gametree
function branchingFactor(children list number) returns number
end
function treeDepth(depths list number) returns number
end
function minimaxLeaf(values list number, maxNode bool) returns number
end
function expectimax(values list number, probs list number) returns number
end
function alphaBetaWindow(alpha number, beta number) returns list number
end
function gameTreeSize(branching number, depth number) returns number
end
function plyCount(moves list number) returns number
end
function bestChild(scores list number) returns number
end
end
```